ASSIGNMENT #2

Course: **Compiler Construction**

Instructor: **Ms. Nirmal Tariq**

Ruhab Ahmad **(23I-0559)**

Hasan Altaf **(23I-0698)**

Degree: **Computer Science**

Section: **E**

FAST - NUCES

LL(1) Parser Assignment Report

1. Introduction

LL(1) parsing is a top-down parsing technique that reads input Left-to-right and produces a Leftmost derivation using 1 lookahead token. A grammar is LL(1) if each parsing table cell contains at most one production. This project implements the full LL(1) pipeline from grammar normalization to stack-based parsing with error handling and parse tree generation.

2. Approach

2.1 Data Structures Used

- Rule structure for grammar non-terminals and productions.
- Set structure for FIRST and FOLLOW sets.
- 2D parsing table for LL(1) actions.
- Array-based stack for predictive parsing.
- Tree node structure for parse tree generation.

2.2 Algorithm Implementation Details

1. CFG input parsing:

- Grammar is read line-by-line from file.
- Productions are split using -> and |.
- Epsilon is normalized from @ to epsilon.

2. Left factoring:

- Common token prefixes are detected.
- Grammar is rewritten using prime non-terminals.
- Factoring repeats until no common prefix remains.

3. Left recursion removal:

- Indirect recursion is resolved with ordered substitution Ai, Aj.
- Direct recursion is removed using A and A' transformation.

4. FIRST/FOLLOW sets:

- Fixed-point iteration is used until no changes occur.
- FIRST(beta) and FOLLOW(A) propagation rules are applied.

5. LL(1) table construction:

- Table entries are added from FIRST(alpha).

- FOLLOW(A) is used when epsilon belongs to FIRST(alpha).
- Conflict detection marks grammar as non-LL(1).

6. Stack-based parser:

- Stack initialized as $, StartSymbol.
- Match, expand, and error cases handled according to LL(1) algorithm.
- Step-by-step trace is printed and saved.

2.3 Design Decisions

- Modular file organization was used to separate grammar logic, set computation, parsing logic, stack, tree, and error handling.
- Token-based production handling was chosen to avoid character-level ambiguity.
- Parse traces and parse trees are written to output files for reproducibility.

2.4 Indirect Left Recursion Handling Approach

- Non-terminals are processed in order A1 to An.
- For each Ai, all productions starting with previous Aj are expanded using Aj alternatives.
- After substitution, direct left recursion elimination is applied on Ai.
- New helper non-terminals use prime notation (for example Expr').

2.5 Error Recovery Strategy Chosen

Panic-mode inspired recovery was implemented:

- Empty table entry on non-terminal: If lookahead is in FOLLOW, pop stack; otherwise skip input.
- Terminal mismatch: Report missing symbol and pop stack symbol.
- Extra trailing input: Report unexpected symbol and skip input token.
- Continuous parsing to detect multiple errors in one input string.

3. Challenges

- **Challenge: Keeping transformations correct after modular split**
  Solution: Reused same logic and preserved data flow across modules.
- **Challenge: Correct parse tree construction**
  Solution: Children are created in left-to-right order, then pushed right-to-left.
- **Challenge: Avoiding parser stop at first error**
  Solution: Added recovery branches for non-terminal and terminal mismatch cases.
- **Challenge: Step trace clarity**
  Solution: Printed stack, remaining input, and action at each parser step.

4. Test Cases

4.1 Grammar Set

- 1. Grammar 1 (Expression): Expr -> Expr + Term | Term; Term -> Term * Factor | Factor; Factor -> (Expr) | id
- 2. Grammar 2 (Indirect Recursion): Start -> Alpha a | b; Alpha -> Start c | d
- 3. Grammar 3 (If-Then-Else): Stmt -> if Expr then Stmt else Stmt | if Expr then Stmt | id; Expr -> id

5. Verification

- Build validation using Makefile.
- Manual review of transformed grammar.
- FIRST/FOLLOW consistency checks.
- LL(1) table conflict checks.
- Parsing trace inspection.
- Error logs and parse tree validation.

6. Sample Outputs

Task 1.1

```
Original Grammar (Task 1.1)
Expr -> Expr + Term | Term
Term -> Term * Factor | Factor
Factor -> ( Expr ) | id

After Left Factoring (Task 1.2)
Expr -> Expr + Term | Term
Term -> Term * Factor | Factor
Factor -> ( Expr ) | id

After Left Recursion Removal (Task 1.3)
Expr -> Term Expr'
Term -> Factor Term'
Factor -> ( Expr ) | id
Expr' -> + Term Expr' | epsilon
Term' -> * Factor Term' | epsilon

FIRST Sets (Tabular)
Non-Terminal        FIRST
------------        -----
Expr                { (, id }
Term                { (, id }
Factor              { (, id }
Expr'               { +, epsilon }
Term'               { *, epsilon }

FOLLOW Sets (Tabular)
Non-Terminal        FOLLOW
------------        ------
Expr                { ), $ }
Term                { +, ), $ }
Factor              { *, +, ), $ }
Expr'               { ), $ }
Term'               { +, ), $ }
```

```
LL(1) Parsing Table
NT/T              | (               | )               | id              | +               | *               | $
Expr              | Term Expr'      |                 | Term Expr'      |                 |                 |
Term              | Factor Term'    |                 | Factor Term'    |                 |                 |
Factor            | ( Expr )        |                 | id              |                 |                 |
Expr'             |                 | epsilon         |                 | + Term Expr'    |                 | epsilon
Term'             |                 | epsilon         |                 | epsilon         | * Factor Term'  | epsilon

Grammar Status: LL(1)

=== Input Line 1 ===
Input: id + id * id
Step   | Stack(bottom->top)                    | Remaining Input                       | Action
2      | $ Expr                                | id + id * id $                        | Expand: Expr -> Term Expr'
4      | $ Expr' Term                          | id + id * id $                        | Expand: Term -> Factor Term'
6      | $ Expr' Term' Factor                  | id + id * id $                        | Expand: Factor -> id
8      | $ Expr' Term' id                      | id + id * id $                        | Match 'id'
10     | $ Expr' Term'                         | + id * id $                           | Expand: Term' -> epsilon
12     | $ Expr'                               | + id * id $                           | Expand: Expr' -> + Term Expr'
14     | $ Expr' Term +                        | + id * id $                           | Match '+'
16     | $ Expr' Term                          | id * id $                             | Expand: Term -> Factor Term'
18     | $ Expr' Term' Factor                  | id * id $                             | Expand: Factor -> id
20     | $ Expr' Term' id                      | id * id $                             | Match 'id'
22     | $ Expr' Term'                         | * id $                                | Expand: Term' -> * Factor Term'
24     | $ Expr' Term' Factor *                | * id $                                | Match '*'
26     | $ Expr' Term' Factor                  | id $                                  | Expand: Factor -> id
28     | $ Expr' Term' id                      | id $                                  | Match 'id'
30     | $ Expr' Term'                         | $                                     | Expand: Term' -> epsilon
32     | $ Expr'                               | $                                     | Expand: Expr' -> epsilon
34     | $                                     | $                                     | ACCEPT
Result: ACCEPTED
```

```
=== Input Line 2 ===
Input: ( id + id * id
Step   | Stack(bottom->top)                    | Remaining Input                       | Action
1      | $ Expr                                | ( id + id * id $                      | Expand: Expr -> Term Expr'
2      | $ Expr' Term                          | ( id + id * id $                      | Expand: Term -> Factor Term'
3      | $ Expr' Term' Factor                  | ( id + id * id $                      | Expand: Factor -> ( Expr )
4      | $ Expr' Term' ) Expr (                | ( id + id * id $                      | Match '('
5      | $ Expr' Term' ) Expr                  | id + id * id $                        | Expand: Expr -> Term Expr'
6      | $ Expr' Term' ) Expr' Term            | id + id * id $                        | Expand: Term -> Factor Term'
7      | $ Expr' Term' ) Expr' Term' Factor    | id + id * id $                        | Expand: Factor -> id
8      | $ Expr' Term' ) Expr' Term' id        | id + id * id $                        | Match 'id'
9      | $ Expr' Term' ) Expr' Term'           | + id * id $                           | Expand: Term' -> epsilon
10     | $ Expr' Term' ) Expr'                 | + id * id $                           | Expand: Expr' -> + Term Expr'
11     | $ Expr' Term' ) Expr' Term +          | + id * id $                           | Match '+'
12     | $ Expr' Term' ) Expr' Term            | id * id $                             | Expand: Term -> Factor Term'
13     | $ Expr' Term' ) Expr' Term' Factor    | id * id $                             | Expand: Factor -> id
14     | $ Expr' Term' ) Expr' Term' id        | id * id $                             | Match 'id'
15     | $ Expr' Term' ) Expr' Term'           | * id $                                | Expand: Term' -> * Factor Term'
16     | $ Expr' Term' ) Expr' Term' Factor *  | * id $                                | Match '*'
17     | $ Expr' Term' ) Expr' Term' Factor    | id $                                  | Expand: Factor -> id
18     | $ Expr' Term' ) Expr' Term' id        | id $                                  | Match 'id'
19     | $ Expr' Term' ) Expr' Term'           | $                                     | Expand: Term' -> epsilon
20     | $ Expr' Term' ) Expr'                 | $                                     | Expand: Expr' -> epsilon
[Parse Error] line=2, column=7, type=PrematureEnd, expected=), found=$
21     | $ Expr' Term' )                       | $                                     | Recovery: insert missing ')' (pop stack)
22     | $ Expr' Term'                         | $                                     | Expand: Term' -> epsilon
23     | $ Expr'                               | $                                     | Expand: Expr' -> epsilon
24     | $                                     | $                                     | ACCEPT
Result: COMPLETED WITH 1 ERROR(S)
```

```
=== Input Line 3 ===
Input: id id + )
Step   | Stack(bottom->top)                    | Remaining Input                       | Action
1      | $ Expr                                | id id + ) $                           | Expand: Expr -> Term Expr'
2      | $ Expr' Term                          | id id + ) $                           | Expand: Term -> Factor Term'
3      | $ Expr' Term' Factor                  | id id + ) $                           | Expand: Factor -> id
4      | $ Expr' Term' id                      | id id + ) $                           | Match 'id'
[Parse Error] line=3, column=2, type=EmptyTableEntry, expected=), +, *, $, found=id
5      | $ Expr' Term'                         | id + ) $                              | Panic recovery: skip input token 'id'
6      | $ Expr' Term'                         | + ) $                                 | Expand: Term' -> epsilon
7      | $ Expr'                               | + ) $                                 | Expand: Expr' -> + Term Expr'
8      | $ Expr' Term +                        | + ) $                                 | Match '+'
[Parse Error] line=3, column=4, type=EmptyTableEntry, expected=(, id, found=)
9      | $ Expr' Term                          | ) $                                   | Panic recovery: pop Term (sync by FOLLOW)
10     | $ Expr'                               | ) $                                   | Expand: Expr' -> epsilon
[Parse Error] line=3, column=4, type=UnexpectedSymbol, expected=$, found=)
11     | $                                     | ) $                                   | Recovery: skip extra input ')'
12     | $                                     | $                                     | ACCEPT
Result: COMPLETED WITH 3 ERROR(S)
```

```
Expr
  Term
    Factor
      id
    Term'
      epsilon
  Expr'
    +
    Term
      Factor
        id
      Term'
        *
        Factor
          id
        Term'
          epsilon
    Expr'
      epsilon

=== Input Line 2 ===
Input: ( id + id ) * id
Step   | Stack(bottom->top)                    | Remaining Input                 | Action
2      | $ Expr                                | ( id + id ) * id $              | Expand: Expr -> Term Expr'
4      | $ Expr' Term                          | ( id + id ) * id $              | Expand: Term -> Factor Term'
6      | $ Expr' Term' Factor                 | ( id + id ) * id $              | Expand: Factor -> ( Expr )
8      | $ Expr' Term' ) Expr (               | ( id + id ) * id $              | Match '('
10     | $ Expr' Term' ) Expr                | id + id ) * id $                | Expand: Expr -> Term Expr'
12     | $ Expr' Term' ) Expr' Term           | id + id ) * id $                | Expand: Term -> Factor Term'
14     | $ Expr' Term' ) Expr' Term' Factor   | id + id ) * id $                | Expand: Factor -> id
16     | $ Expr' Term' ) Expr' Term' id       | id + id ) * id $                | Match 'id'
18     | $ Expr' Term' ) Expr' Term'          | + id ) * id $                   | Expand: Term' -> epsilon
20     | $ Expr' Term' ) Expr'                | + id ) * id $                   | Expand: Expr' -> + Term Expr'
22     | $ Expr' Term' ) Expr' Term +         | + id ) * id $                   | Match '+'
24     | $ Expr' Term' ) Expr' Term           | id ) * id $                     | Expand: Term -> Factor Term'
26     | $ Expr' Term' ) Expr' Term' Factor   | id ) * id $                     | Expand: Factor -> id
28     | $ Expr' Term' ) Expr' Term' id       | id ) * id $                     | Match 'id'
30     | $ Expr' Term' ) Expr' Term'          | ) * id $                        | Expand: Term' -> epsilon
30     | $ Expr' Term' ) Expr' Term'          | ) * id $                        | Expand: Term' -> epsilon
32     | $ Expr' Term' ) Expr'                | ) * id $                        | Expand: Expr' -> epsilon
34     | $ Expr' Term' )                      | ) * id $                        | Match ')'
36     | $ Expr' Term'                        | * id $                          | Expand: Term' -> * Factor Term'
38     | $ Expr' Term' Factor *               | * id $                          | Match '*'
40     | $ Expr' Term' Factor                 | id $                            | Expand: Factor -> id
42     | $ Expr' Term' id                     | id $                            | Match 'id'
44     | $ Expr' Term'                        | $                               | Expand: Term' -> epsilon
46     | $ Expr'                              | $                               | Expand: Expr' -> epsilon
48     | $                                    | $                               | ACCEPT
Result: ACCEPTED
Expr
  Term
    Factor
      (
      Expr
        Term
          Factor
            id
          Term'
            epsilon
        Expr'
          +
          Term
            Factor
              id
            Term'
              epsilon
          Expr'
            epsilon
      )
    Term'
      *
      Factor
        id
      Term'
        epsilon
  Expr'
```

```
=== Input Line 3 ===
Input: id * id + id
Step   | Stack(bottom->top)                | Remaining Input                | Action
2      | $ Expr                            | id * id + id $                 | Expand: Expr -> Term Expr'
4      | $ Expr' Term                      | id * id + id $                 | Expand: Term -> Factor Term'
6      | $ Expr' Term' Factor              | id * id + id $                 | Expand: Factor -> id
8      | $ Expr' Term' id                 | id * id + id $                 | Match 'id'
10     | $ Expr' Term'                    | * id + id $                    | Expand: Term' -> * Factor Term'
12     | $ Expr' Term' Factor *           | * id + id $                    | Match '*'
14     | $ Expr' Term' Factor             | id + id $                      | Expand: Factor -> id
16     | $ Expr' Term' id                 | id + id $                      | Match 'id'
18     | $ Expr' Term'                    | + id $                         | Expand: Term' -> epsilon
20     | $ Expr'                          | + id $                         | Expand: Expr' -> + Term Expr'
22     | $ Expr' Term +                   | + id $                         | Match '+'
24     | $ Expr' Term                     | id $                           | Expand: Term -> Factor Term'
26     | $ Expr' Term' Factor             | id $                           | Expand: Factor -> id
28     | $ Expr' Term' id                 | id $                           | Match 'id'
30     | $ Expr' Term'                    | $                              | Expand: Term' -> epsilon
32     | $ Expr'                          | $                              | Expand: Expr' -> epsilon
34     | $                                | $                              | ACCEPT
Result: ACCEPTED
Expr
  Term
    Factor
      id
    Term'
      *
      Factor
        id
      Term'
        epsilon
  Expr'
    +
    Term
      Factor
        id
      Term'
```

7. Conclusion

This implementation demonstrates the full LL(1) parsing workflow including grammar transformation, set computation, table construction, and predictive parsing. The project reinforced key compiler concepts and provided experience in designing stable, informative parsers.